

目录

PART1_ESP32 + LVGL 智能面板 1

一、继电器控制（真实灯光开关）1

二、PWM 调光（真实亮度调节）3

三、MQTT 远程控制（接入云平台）5

四、OTA 空中升级（WiFi 远程更新固件）8

五、LVGL 多页面切换10

六、完整项目架构总结12

七、生产级项目注意事项13

PART2_ESP32 + LVGL 智能控制面板完整工程结构 13

一、工程目录结构13

二、各文件核心内容说明14

三、日常使用操作命令18

四、项目维护与版本管理19

五、备份与安全21

六、功能迭代开发流程22

七、常见维护问题处理24

八、量产前检查清单25

PART3_ESP32 智能面板完整项目25

第一步：解压并配置26

第二步：菜单配置27

第三步：编译烧录27

1. 代码版本管理27

2. 版本号管理28

3. 配置安全28

4. 常见维护操作28

PART1_ESP32 + LVGL 智能面板

一、继电器控制（真实灯光开关）

原理

继电器是电控开关，ESP32 输出高低电平控制继电器线圈通断，进而控制 220V 强电通断。

接线

继电器模块	ESP32	说明
-------	-------	----

VCC	5V / 3.3V	看模块规格，大部分是5V
GND	GND	共地
IN	GPIO26	控制信号
COM	火线进线	接 220V 火线
NO（常开）	火线出线	接灯泡一端

⚠ 安全警告：220V 强电危险，接线务必断电操作，建议新手先用 LED 灯代替强电测试。

核心代码

```
C
#include 'driver/gpio.h'

#define RELAY_PIN  GPIO_NUM_26

// 初始化继电器
void relay_init(void) {
    gpio_set_direction(RELAY_PIN, GPIO_MODE_OUTPUT);
    gpio_set_level(RELAY_PIN, 0); // 默认断开
}

// 控制继电器：1=闭合通电，0=断开
void relay_set(bool on) {
    gpio_set_level(RELAY_PIN, on ? 1 : 0);
}
```

集成到开关事件

修改之前的 `light_switch_cb`:

```
C
static void light_switch_cb(lv_event_t *e) {
    lv_obj_t *sw = lv_event_get_target(e);
    bool state = lv_obj_has_state(sw, LV_STATE_CHECKED);

    if (sw == sw_light1) {
        relay_set(state); // 控制真实继电器
    }
}
```

二、PWM 调光（真实亮度调节）

原理

ESP32 的 **LEDC** 外设输出 PWM 方波，通过改变占空比调节 LED 平均电压，实现亮度变化。

接线

LED 正极 → 串联 220Ω 电阻 → GPIO27

LED 负极 → GND

核心代码

```
C
#include 'driver/ledc.h'

#define LEDC_TIMER    LEDC_TIMER_0
```

```

#define LEDC_MODE      LEDC_LOW_SPEED_MODE
#define LEDC_CHANNEL    LEDC_CHANNEL_0
#define LEDC_GPIO      GPIO_NUM_27
#define LEDC_FREQ      5000    // 5kHz 频率，无频闪
#define LEDC_RESOLUTION LEDC_TIMER_13_BIT // 13 位分辨率（0~8191）

// 初始化 PWM
void pwm_led_init(void) {
    // 配置定时器
    ledc_timer_config_t timer_cfg = {
        .speed_mode = LEDC_MODE,
        .timer_num = LEDC_TIMER,
        .freq_hz = LEDC_FREQ,
        .duty_resolution = LEDC_RESOLUTION,
    };
    ledc_timer_config(&timer_cfg);

    // 配置通道
    ledc_channel_config_t ch_cfg = {
        .gpio_num = LEDC_GPIO,
        .speed_mode = LEDC_MODE,
        .channel = LEDC_CHANNEL,
        .timer_sel = LEDC_TIMER,
        .duty = 0,        // 初始占空比
        .hpoint = 0,
    };
    ledc_channel_config(&ch_cfg);
}

// 设置亮度：0~100 百分比
void pwm_set_brightness(int percent) {
    // 百分比 → 占空比数值（13 位：0~8191）

```

```
uint32_t duty = (uint32_t)(percent * 8191 / 100);  
ledc_set_duty(LED0_CHANNEL, LED0_CHANNEL, duty);  
ledc_update_duty(LED0_CHANNEL, LED0_CHANNEL);  
}
```

集成到滑块事件

```
C  
static void brightness_slider_cb(lv_event_t *e) {  
    int val = lv_slider_get_value(lv_event_get_target(e));  
    lv_label_set_text_fmt(label_brightness_val, '%d%%', val);  
    pwm_set_brightness(val); // 真实调光  
}
```

三、MQTT 远程控制（接入云平台）

原理

ESP32 连接 WiFi 后，通过 MQTT 协议与云端通信，实现手机 App 远程控制开关、上报温湿度数据。

核心组件

- **MQTT Broker:** 公共测试用 `mqtt.eclipseprojects.io`，生产用阿里云/EMQX
- 发布 **Topic:** `device/sensor/data`（上报数据）
- 订阅 **Topic:** `device/control/light`（接收控制指令）

代码实现

```
C
```

```

#include 'mqtt_client.h'

static esp_mqtt_client_handle_t mqtt_client;

// MQTT 事件回调
static void mqtt_event_handler(void *handler_args, esp_event_base_t base,
                                int32_t event_id, void *event_data) {
    esp_mqtt_event_handle_t event = event_data;

    switch ((esp_mqtt_event_id_t)event_id) {
        case MQTT_EVENT_CONNECTED:
            printf('MQTT 已连接\n');
            // 订阅控制主题
            esp_mqtt_client_subscribe(mqtt_client, 'device/control/light', 0);
            break;

        case MQTT_EVENT_DATA:
            // 收到控制消息
            printf('收到指令: topic=%.1s, data=%.1s\n',
                    event->topic_len, event->topic,
                    event->data_len, event->data);

            // 解析控制指令（简单示例：1=开灯，0=关灯）
            if (strncmp(event->data, '1', 1) == 0) {
                relay_set(true);
                lv_obj_add_state(sw_light1, LV_STATE_CHECKED);
            } else {
                relay_set(false);
                lv_obj_clear_state(sw_light1, LV_STATE_CHECKED);
            }
            break;
    }
}

```

```

        default: break;
    }
}

// 初始化 MQTT
void mqtt_app_start(void) {
    esp_mqtt_client_config_t mqtt_cfg = {
        .broker.address.uri = 'mqtt://mqtt.eclipseprojects.io',
    };

    mqtt_client = esp_mqtt_client_init(&mqtt_cfg);
    esp_mqtt_client_register_event(mqtt_client, ESP_EVENT_ANY_ID,
    mqtt_event_handler, NULL);
    esp_mqtt_client_start(mqtt_client);
}

// 上报温湿度数据
void mqtt_publish_sensor(float temp, float humi) {
    char payload[64];
    snprintf(payload, sizeof(payload),
        '{\'temp\':%.1f, \'humi\':%.0f}', temp, humi);
    esp_mqtt_client_publish(mqtt_client, 'device/sensor/data',
        payload, 0, 1, 0);
}

```

使用方法

1. WiFi 连接成功后调用 `mqtt_app_start()`
 2. 数据更新任务中定时调用 `mqtt_publish_sensor()` 上报
 3. 手机端用 MQTT 工具（如 MQTT Explorer）发指令远程控制
-

四、OTA 空中升级（WiFi 远程更新固件）

原理

ESP32 通过 HTTP/HTTPS 下载新固件到空闲 Flash 分区，校验通过后切换分区重启，完成升级。

前提

- Flash 分区表使用 factory + OTA 模式
- 服务器上存放新固件 .bin 文件

核心代码

```
C
#include 'esp_ota_ops.h'
#include 'esp_http_client.h'

esp_err_t ota_update(const char *url) {
    esp_http_client_config_t config = {
        .url = url,
        .timeout_ms = 60000,
    };

    esp_ota_handle_t update_handle = 0;
    const esp_partition_t *update_part =
    esp_ota_get_next_update_partition(NULL);

    printf('开始 OTA 升级: %s\n', url);

    esp_http_client_handle_t client = esp_http_client_init(&config);
    esp_http_client_open(client, 0);
    int content_len = esp_http_client_fetch_headers(client);
```

```

    esp_ota_begin(update_part, OTA_SIZE_UNKNOWN, &update_handle);

    char buf[1024];
    int total_read = 0;
    while (total_read < content_len) {
        int read_len = esp_http_client_read(client, buf, sizeof(buf));
        if (read_len <= 0) break;
        esp_ota_write(update_handle, buf, read_len);
        total_read += read_len;
    }

    esp_ota_end(update_handle);
    esp_ota_set_boot_partition(update_part);
    esp_http_client_cleanup(client);

    printf('升级完成，3 秒后重启\n');
    vTaskDelay(pdMS_TO_TICKS(3000));
    esp_restart();

    return ESP_OK;
}

```

集成到 UI

添加一个"检查更新"按钮，点击触发 OTA:

```

C
// 在 create_ui 中添加
lv_obj_t *btn_ota = lv_btn_create(lv_scr_act());
lv_obj_t *label_ota = lv_label_create(btn_ota);
lv_label_set_text(label_ota, '检查更新');

```

```
lv_obj_add_event_cb(btn_ota, ota_btn_cb, LV_EVENT_CLICKED, NULL);

// 回调函数
static void ota_btn_cb(lv_event_t *e) {
    xTaskCreate(ota_update_task, 'ota', 8192, NULL, 5, NULL);
}

void ota_update_task(void *pv) {
    ota_update('http://你的服务器/firmware_v2.bin');
    vTaskDelete(NULL);
}
```

五、LVGL 多页面切换

原理

每个页面是一个独立的 `lv_obj`（屏幕对象），用 `lv_scr_load()` 切换显示。

代码实现

```
C
// 全局页面对象

static lv_obj_t *scr_main; // 主页
static lv_obj_t *scr_setting; // 设置页
static lv_obj_t *scr_about; // 关于页

// 创建主页
static void create_scr_main(void) {
    scr_main = lv_obj_create(NULL);

    // ... 主页内容（之前的控制面板代码放这里）
}
```

```

// 跳转到设置页的按钮

lv_obj_t *btn = lv_btn_create(scr_main);

lv_obj_align(btn, LV_ALIGN_BOTTOM_RIGHT, -10, -10);

lv_obj_t *label = lv_label_create(btn);

lv_label_set_text(label, '设置');

lv_obj_add_event_cb(btn, goto_setting_cb, LV_EVENT_CLICKED, NULL);
}

// 创建设置页

static void create_scr_setting(void) {

    scr_setting = lv_obj_create(NULL);

    lv_obj_t *title = lv_label_create(scr_setting);

    lv_label_set_text(title, '设置');

    lv_obj_align(title, LV_ALIGN_TOP_MID, 0, 20);

    // WiFi 设置项、亮度设置项...

    // 返回主页按钮

    lv_obj_t *btn_back = lv_btn_create(scr_setting);

    lv_obj_align(btn_back, LV_ALIGN_BOTTOM_LEFT, 10, -10);

    lv_obj_t *label = lv_label_create(btn_back);

    lv_label_set_text(label, '返回');

    lv_obj_add_event_cb(btn_back, goto_main_cb, LV_EVENT_CLICKED, NULL);
}

// 页面切换回调

static void goto_setting_cb(lv_event_t *e) {

    lv_scr_load_anim(scr_setting, LV_SCR_LOAD_ANIM_MOVE_LEFT, 200, 0, false);
}

```

```
static void goto_main_cb(lv_event_t *e) {  
    lv_scr_load_anim(scr_main, LV_SCR_LOAD_ANIM_MOVE_RIGHT, 200, 0, false);  
}
```

app_main 中初始化

```
C  
// 创建所有页面  
create_scr_main();  
create_scr_setting();  
  
// 默认加载主页  
lv_scr_load(scr_main);
```

六、完整项目架构总结

Plain Text

app_main

- ├─ 硬件初始化
 - | ├─ GPIO（继电器）
 - | ├─ LEDC（PWM 调光）
 - | └─ SPI/LCD（屏幕显示）
- ├─ 系统组件
 - | ├─ NVS Flash
 - | ├─ WiFi 连接
 - | └─ MQTT 客户端
- ├─ LVGL 图形
 - | ├─ 多页面管理
 - | ├─ 控件事件回调
 - | └─ 数据显示更新
- └─ 业务任务

- └─ 传感器采集任务
- └─ 数据上报任务
- └─ OTA 升级任务（按需触发）

七、生产级项目注意事项

1. 看门狗：启用任务看门狗，防止程序跑飞
2. 异常恢复：WiFi/MQTT 断开自动重连
3. 数据掉电保存：NVS 存储配置参数（WiFi 密码、亮度设置等）
4. 低功耗优化：屏幕常亮功耗大，闲置自动熄屏
5. 固件分区：OTA 双分区，升级失败自动回滚
6. 硬件保护：继电器加续流二极管、光耦隔离

需要我把这些功能整合成一个完整的工程文件结构，或者针对某一个功能再深入展开吗？

PART2_ESP32 + LVGL 智能控制面板完整工程结构

一、工程目录结构

Plain Text

```
esp32_smart_panel/      # 项目根目录
└─ CMakeLists.txt      # 顶层 CMake 配置
└─ sdkconfig            # 编译配置（menuconfig 自动生成）
└─ sdkconfig.defaults   # 默认配置（提交到 Git）
└─ partitions.csv       # Flash 分区表（OTA 需要）
└─ README.md            # 项目说明文档
|
└─ main/                # 主程序目录
|   └─ CMakeLists.txt
|   └─ app_main.c       # 入口文件（初始化+主循环）
```

```
| |—— wifi_connect.c/h # WiFi 连接管理
| |—— mqtt_client.c/h # MQTT 通信
| |—— ota_update.c/h # OTA 空中升级
| |
| |—— hardware/ # 硬件驱动层
| | |—— relay.c/h # 继电器控制
| | |—— pwm_led.c/h # PWM 调光
| | |—— sensor.c/h # 温湿度传感器
| |
| |—— ui/ # UI 界面层
| | |—— ui_manager.c/h # 页面管理（切换逻辑）
| | |—— scr_main.c/h # 主控制面板
| | |—— scr_setting.c/h # 设置页面
| | |—— scr_about.c/h # 关于页面
|
|—— components/ # 第三方组件
| |—— lvgl/ # LVGL 图形库
| |—— lvgl_esp32_drivers/ # ESP32 显示+触摸驱动
| |—— sht30/ # 传感器驱动（示例）
```

二、各文件核心内容说明

1. 顶层 CMakeLists.txt

```
CMake
cmake_minimum_required(VERSION 3.16)

include( ENV {IDF_PATH} /tools/cmake/project.cmake)

project(esp32_smart_panel)
```

2. main/CMakeLists.txt

```
CMake
idf_component_register(
    SRCS
        'app_main.c'
        'wifi_connect.c'
        'mqtt_client.c'
        'ota_update.c'
        'hardware/relay.c'
        'hardware/pwm_led.c'
        'hardware/sensor.c'
        'ui/ui_manager.c'
        'ui/scr_main.c'
        'ui/scr_setting.c'
        'ui/scr_about.c'
    INCLUDE_DIRS
        '.'
        'hardware'
        'ui'
    REQUIRES
        lvgl
        esp_wifi
        mqtt
        esp_http_client
        app_update
)
```

3. app_main.c（入口文件）

只做初始化调度，不写业务逻辑：

C

```
#include 'wifi_connect.h'
```

```
#include 'mqtt_client.h'
```

```
#include 'relay.h'
```

```
#include 'pwm_led.h'
```

```
#include 'sensor.h'
```

```
#include 'ui_manager.h'
```

```
void app_main(void) {
```

```
    // 1. 硬件初始化
```

```
    relay_init();
```

```
    pwm_led_init();
```

```
    sensor_init();
```

```
    // 2. 系统初始化
```

```
    nvs_flash_init();
```

```
    wifi_init();
```

```
    mqtt_start();
```

```
    // 3. UI 初始化（含 LVGL+驱动+所有页面）
```

```
    ui_init();
```

```
    // 4. 启动业务任务
```

```
    xTaskCreate(sensor_task, 'sensor', 2048, NULL, 4, NULL);
```

```
    xTaskCreate(data_report_task, 'report', 4096, NULL, 3, NULL);
```

```
    // 5. LVGL 主循环
```

```
    while (1) {
```

```
        lv_timer_handler();
```

```
        vTaskDelay(pdMS_TO_TICKS(5));
```

```
    }
```

```
}
```

4. ui/ui_manager.c (页面管理器)

统一管理所有页面的创建和切换:

```
C
#include 'ui_manager.h'
#include 'scr_main.h'
#include 'scr_setting.h'
#include 'scr_about.h'

static lv_obj_t *current_scr;

void ui_init(void) {
    lv_init();
    lvgl_driver_init();

    // 显示缓冲和驱动注册... (之前讲过的代码)
    // 触摸输入设备注册...

    // 创建所有页面
    scr_main_create();
    scr_setting_create();
    scr_about_create();

    // 默认显示主页
    ui_switch_to(SCREEN_MAIN);
}

void ui_switch_to(screen_id_t id) {
    switch (id) {
        case SCREEN_MAIN:
            lv_scr_load_anim(scr_main, LV_SCR_LOAD_ANIM_MOVE_RIGHT, 200, 0,
```

```
false);  
    break;  
    case SCREEN_SETTING:  
        lv_scr_load_anim(scr_setting, LV_SCR_LOAD_ANIM_MOVE_LEFT, 200, 0,  
false);  
        break;  
    case SCREEN_ABOUT:  
        lv_scr_load_anim(scr_about, LV_SCR_LOAD_ANIM_MOVE_LEFT, 200, 0,  
false);  
        break;  
    }  
}
```

5. partitions.csv (OTA 双分区表)

Plain Text

```
# Name, Type, SubType, Offset, Size, Flags  
nvs, data, nvs, 0x9000, 0x6000,  
phy_init, data, phy, 0xf000, 0x1000,  
factory, app, factory, 0x10000, 0x1E0000,  
ota_0, app, ota_0, 0x200000, 0x1E0000,  
ota_1, app, ota_1, 0x3E0000, 0x1E0000,
```

三、日常使用操作命令

1. 编译烧录

Bash

```
# 配置项目（第一次或改硬件时用）
idf.py menuconfig

# 编译
idf.py build

# 烧录+打开串口（一条龙）
idf.py -p /dev/ttyUSB0 flash monitor

# 仅擦除 Flash（换项目或出奇怪问题时）
idf.py -p /dev/ttyUSB0 erase_flash
```

2. 调试常用

```
Bash
# 查看固件大小
idf.py size

# 查看各组件内存占用
idf.py size-components

# 生成反汇编（排查崩溃用）
idf.py app

xtensa-esp32-elf-objdump -d build/main/main.elf | app_disasm.txt
```

四、项目维护与版本管理

1. Git 版本管理（必做）

初始化仓库

```
Bash
cd esp32_smart_panel
git init
```

.gitignore 文件（必须创建）

```
Plain Text
# 编译产物
build/

sdkconfig
sdkconfig.old

*.bin
*.elf
*.map

# 临时文件
*.log

.DS_Store
.vscode/
.idea/

# 自动拉取的组件
managed_components/
```

提交代码

```
Bash
git add .
git commit -m 'feat: 完成主控制面板 UI 和继电器控制'
```

版本号规范

采用 语义化版本：v 主版本.次版本.修订号

- v1.0.0: 首个正式版本
- v1.1.0: 新增设置页面功能
- v1.1.1: 修复 WiFi 断连 bug

2. 代码提交规范

Plain Text

feat: 新增功能

fix: 修复 bug

refactor: 重构代码

docs: 文档更新

perf: 性能优化

test: 测试相关

五、备份与安全

1. 多备份策略

备份方式	用途	频率
Git 远程仓库 (Gitee/GitHub)	代码版本管理	每次功能完成
本地完整压缩包	灾难恢复	每月一次
固件 bin 文件归档	版本回退	每次发布

2. 固件归档

每次发布量产版本，保留对应 **bin** 文件：

```
Plain Text
firmware_releases/
├── v1.0.0/
│   ├── smart_panel_v1.0.0.bin
│   └── release_notes.md
├── v1.1.0/
└── ...
```

3. 敏感信息保护

不要把 **WiFi** 密码、**MQTT** 密钥提交到 **Git**！

创建 `config_local.h` 放本地配置，加入 `.gitignore`：

```
C
// config_local.h（本地配置，不提交 Git）
#define WIFI_SSID '家里的 WiFi'
#define WIFI_PASS '12345678'
#define MQTT_TOKEN 'abc123secret'
```

代码中引用：

```
C
#include 'config_local.h'
```

六、功能迭代开发流程

新增一个功能的标准步骤

Plain Text

1. 开新分支

```
git checkout -b feature/add-timer
```

2. 编码实现

- 写驱动层代码（hardware/）
- 写 UI 页面（ui/）
- 在 app_main 注册初始化

3. 编译测试

```
idf.py build flash monitor
```

4. 验证通过后合并

```
git checkout main
git merge feature/add-timer
```

常见迭代方向

阶段	功能	优先级
V1 基础版	继电器控制+手动开关 UI	P0
V1.1	WiFi+MQTT 远程控制	P1
V1.2	温湿度采集+数据上报	P1
V1.3	OTA 升级	P2
V2.0	定时任务+多场景模式	P2
V2.1	语音控制接入	P3

七、常见维护问题处理

1. 编译报错怎么办

```
Bash
# 清理重建（80%的编译问题能解决）

idf.py fullclean

idf.py build

# 组件缺失
rm -rf managed_components

idf.py reconfigure
```

2. 程序崩溃重启

- 打开串口监视器，查看 **Backtrace** 栈回溯
- 用 `idf.py monitor` 自动解析崩溃地址
- 定位到具体函数，逐行排查

3. 换电脑开发

```
Bash
# 1. 安装 ESP-IDF 环境

# 2. Git 克隆项目
git clone xxx/esp32_smart_panel.git

# 3. 拉取依赖组件
cd esp32_smart_panel
```

```
idf.py reconfigure
```

4. 直接编译

```
idf.py build
```

4. ESP-IDF 版本升级

- 大版本升级（v4→v5）：新建分支测试，确认兼容再合并
- 小版本升级：直接升级，一般兼容
- 升级后先 `idf.py fullclean` 再编译

八、量产前检查清单

- ☐ 所有硬件引脚在 `menuconfig` 中配置正确
- ☐ WiFi/MQTT 参数改为生产环境配置
- ☐ 启用看门狗，异常自动重启
- ☐ OTA 功能测试通过（升级+回滚）
- ☐ 固件版本号写入程序，可在关于页查看
- ☐ 关闭所有调试日志 `printf`
- ☐ 功耗测试达标
- ☐ 归档对应版本源码和固件

需要我把这些文件打包成一个可直接编译运行的模板工程吗？

PART3_ESP32 智能面板完整项目

工程已打包完成 ≠

工程包含内容（共 27 个文件）

分层架构

层级	目录	功能
硬件驱动层	main/hardware/	继电器、PWM 调光、温湿度传感器
系统组件层	main/	WiFi 连接、MQTT 通信、OTA 升级
UI 界面层	main/ui/	主页 / 设置页 / 关于页，三页面切换
入口调度	app_main.c	初始化流程 + 任务调度

核心功能

- ✓ LVGL 多页面 UI（主控制面板 + 设置 + 关于）
- ✓ 继电器开关控制（GPIO26）
- ✓ PWM 亮度调节（GPIO27）
- ✓ 温湿度数据展示（模拟数据，可换真实传感器）
- ✓ WiFi 自动连接 + 断线重连
- ✓ MQTT 数据上报 + 远程控制
- ✓ OTA 空中升级（双分区）

后续使用步骤

第一步：解压并配置

```
bash
unzip esp32_smart_panel.zip
cd esp32_smart_panel

# 1. 复制本地配置文件
cp main/config_local_template.h main/config_local.h
```

```
# 编辑填入你的 WiFi 名称密码

# 2. 安装 LVGL 组件
idf.py add-dependency 'lvgl/lvgl^8.4.0'

# 3. 克隆显示驱动组件
cd components
git clone https://github.com/lvgl/lvgl_esp32_drivers.git
cd ..
```

第二步：菜单配置

```
bash
idf.py menuconfig
```

重点配置三项：

1. **Partition Table** → 选 Custom CSV，填 `partitions.csv`
2. **LVGL ESP32 Drivers** → 选屏幕型号 ST7789 + 触摸型号 + 对应引脚
3. **LVGL configuration** → 色深 16 位、分辨率 240×320

第三步：编译烧录

```
bash
idf.py set-target esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

日常维护规范

1. 代码版本管理

```
bash
# 初始化 Git 仓库
```

```
git init
git add .
git commit -m 'init: 工程模板 v1.0'

# 新功能开分支
git checkout -b feature/add-timer

# 开发完成后合并回主分支
```

2. 版本号管理

- 固件版本号在 `main/ui/scr_about.c` 的 `FIRMWARE_VERSION` 宏修改
- 采用语义化版本: `v 主版本.次版本.修订号`
- 每次发布量产版本, 归档对应 `.bin` 固件文件

3. 配置安全

- `config_local.h` 已加入 `.gitignore`, 密码密钥不会提交到 Git
- 换电脑开发只需重新创建该文件即可

4. 常见维护操作

```
bash
# 清理重建 (解决编译怪问题)
idf.py fullclean && idf.py build

# 查看固件大小
idf.py size

# 擦除整片 Flash
idf.py erase_flash
```

扩展开发建议

按模块添加新功能, 每个模块独立 `.c/.h` 文件, 保持分层清晰:

- 加新传感器 → 放 hardware/ 目录
- 加新网络功能 → 放 main/ 根目录
- 加新页面 → 放 ui/ 目录，在 ui_manager.c 注册

需要我针对某个具体模块再深入讲解，或者补充其他功能吗？

|（注：部分内容可能由 AI 生成）